PART 2 - LOOPS AND CONDITIONALS

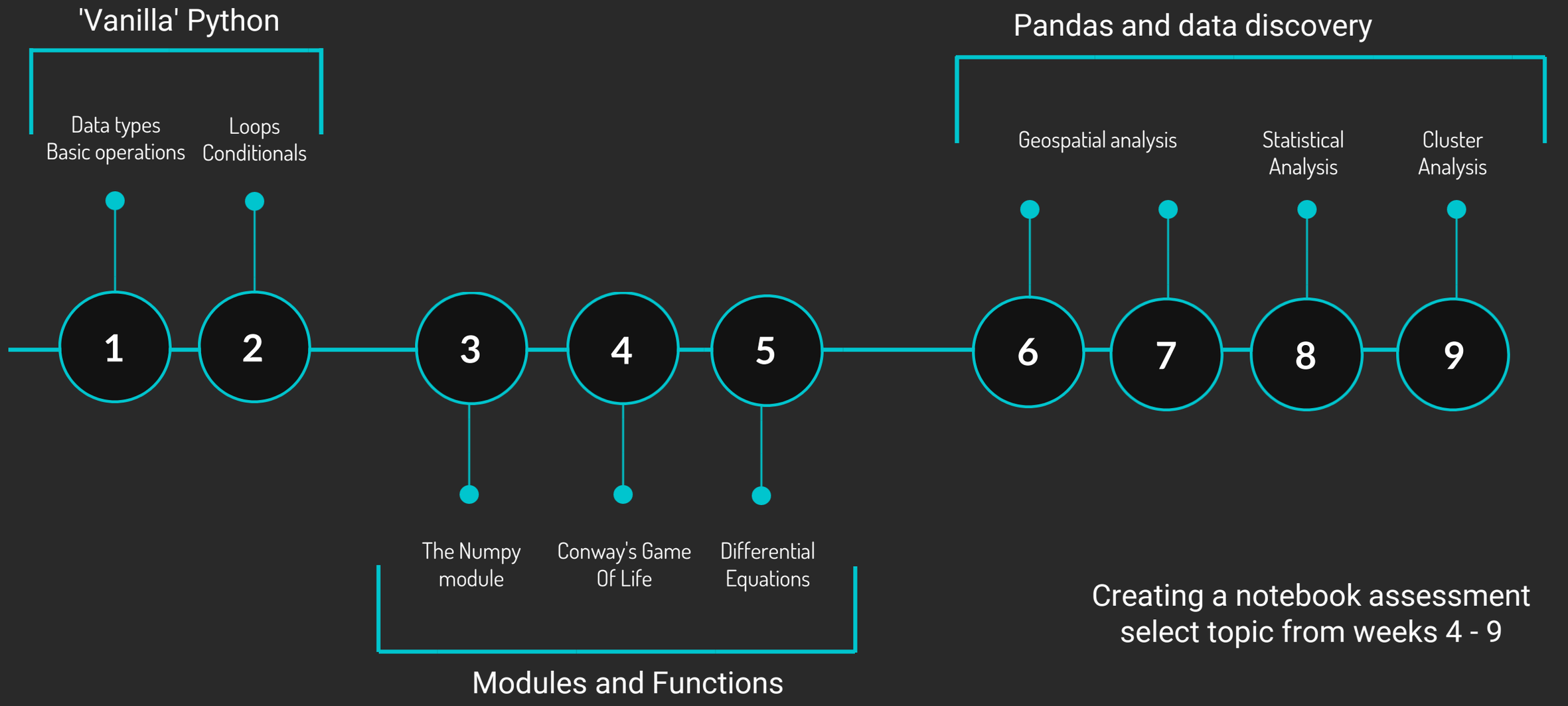


- 'FOR' LOOPS
- CONDITIONAL STATEMENTS
- NESTED LOOPS [LOOPS IN MORE THAN ONE DIMENSION]
- PRACTICAL 2



can you help Bob?

David Topping: david.topping@manchester.ac.uk



COURSE LAYOUT >>>

LECTURE + PRACTICAL. SOLUTIONS ARE GIVEN

RECAP

- Programming is great. Multiple languages, each with their own syntax [set of rules and style]
- We write 'source code' as our software. Code is the text behind our novel!
- We can write python in a seperate .py file, but we will be using Jupyter notebooks in this course

MULTIPLE DATA TYPES

- We can store individual float and integers [and strings...]

```
x = 2.3 # float
y = 4 # integer
z = 5 + 6j # complex
```

- We can store entries in an array, called a list in Vanilla Python, and modify/add entries
- Indexing starts at 0

```
In [7]: # First let's create an identical list and tuple containing names of places
        # Lists use square brackets, whereas tuples use parentheses

        list_capitals = ['London','Paris','Rome','Ankara']
        tuple_capitals = ('London','Paris','Rome','Ankara')

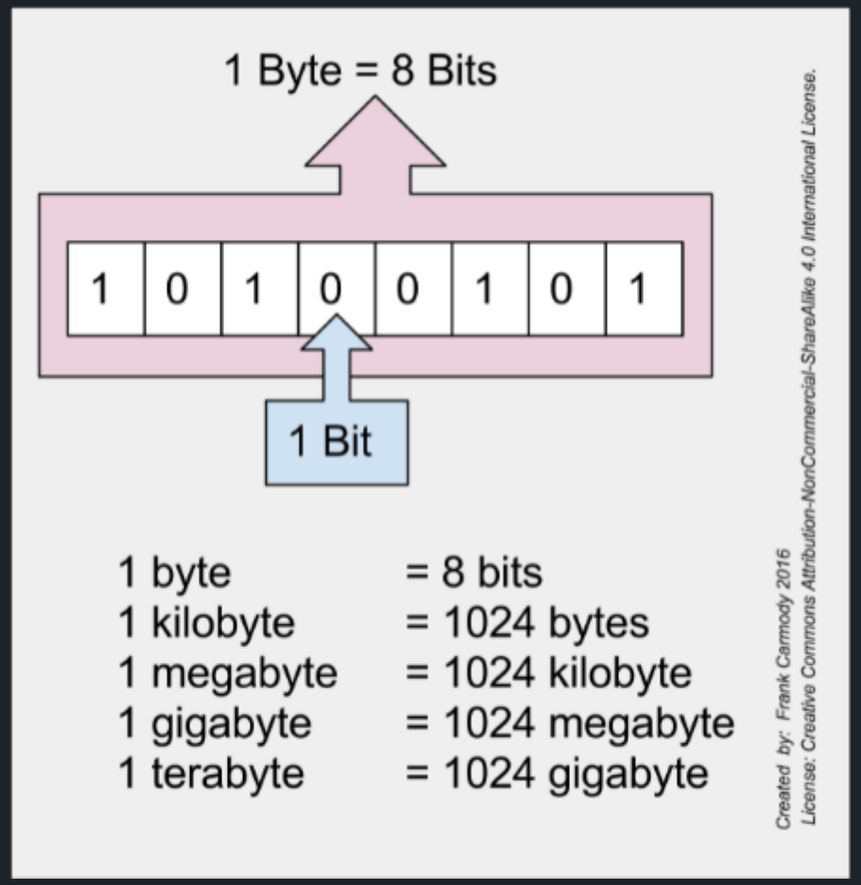
        # If we want to access a particular element from a list of a tuple, we use the same bracket notation but also
        # remember that python indexing starts at 0. Thus, for the first element in the list we would write:

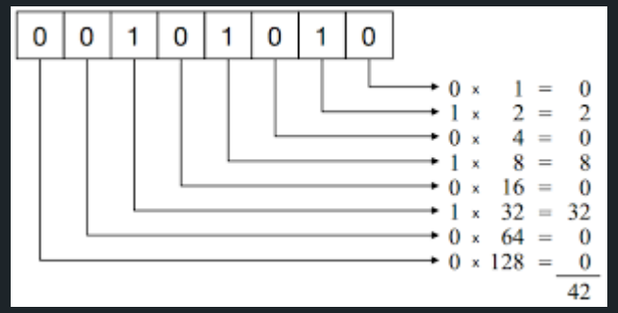
        list_capitals[0]

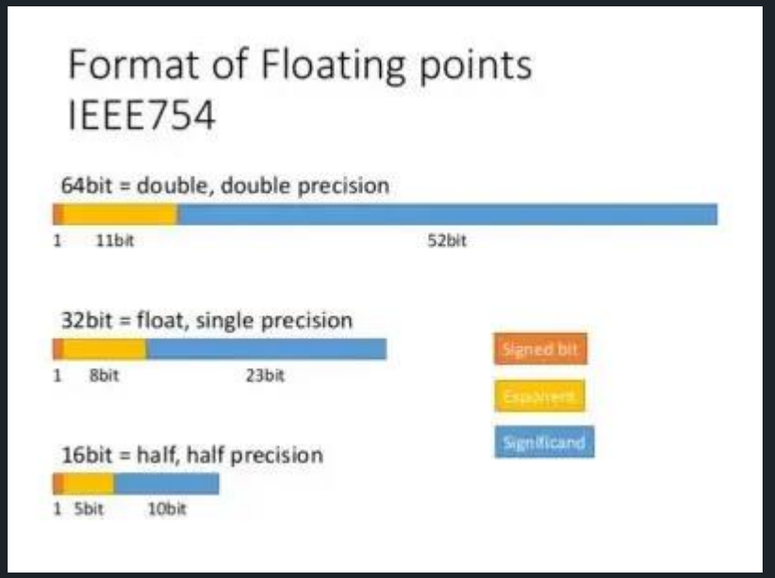
        # Now click run on this portion of code and you should see the word 'London' below, if accessing element [0].

Out[7]: 'London'
```

BEHIND THE SCENES, THE HARDWARE DICTATES THE RANGE OF NUMBERS/WORDS WE CAN WORK WITH







Double precision	0100000000001001001000011111101101010100010001000010110100011000
Single precision	01000000100100100001111111011011
Half precision	0100001001001000

THE THIRD GENERATION OF VIDEO GAME CONSOLES, ALSO REFERRED TO AS THE 8-BIT ERA, BEGAN IN 1983.





RECAP

- Programming is great. Multiple languages, each with their own syntax [set of rules and style]
- We write 'source code' as our software. Code is the text behind our novel!
- We can write python in a seperate .py file, but we will be using Jupyter notebooks in this course

LOOPS

- Rather than manually access values/entries, we can access and modify by ‘looping’ through our arrays

```
In [1]: primes = [2, 3, 5, 7]
        for prime in primes:
            print(prime)

2
3
5
7
```

script.py

```
a_list = [1, 3, 5]

for value in a_list:
    print(value)
```

Output

'FOR' LOOPS

A feature of every programming language. Iterating over components of an array and performing an action or modifying an entry

```
In [1]: primes = [2, 3, 5, 7]
        for prime in primes:
            print(prime)

2
3
5
7
```

```
In [2]: for step in range(7):
            print(step)

0
1
2
3
4
5
6
```

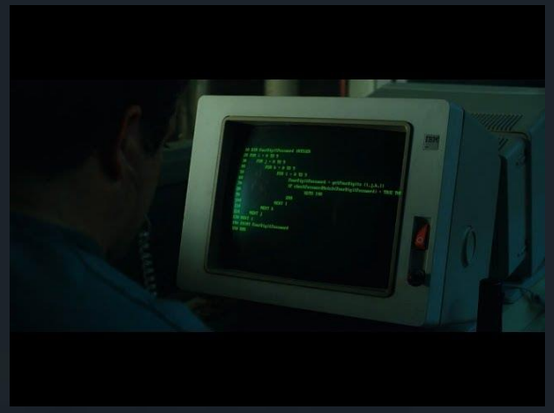


Can be any name I want
The variable adopts the same format as the array/iterable it is searching across

PART 2 - LOOPS AND CONDITIONALS



- 'FOR' LOOPS
- CONDITIONAL STATEMENTS
- NESTED LOOPS [LOOPS IN MORE THAN ONE DIMENSION]
- PRACTICAL 2



can you help Bob?

David Topping: david.topping@manchester.ac.uk

CONDITIONAL STATEMENTS: THE IF, ELIF AND ELSE STATEMENTS

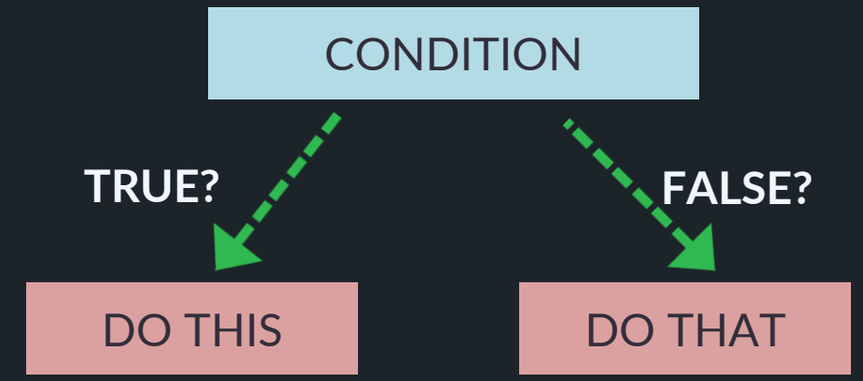
Changing the flow of a program using basic logic very important

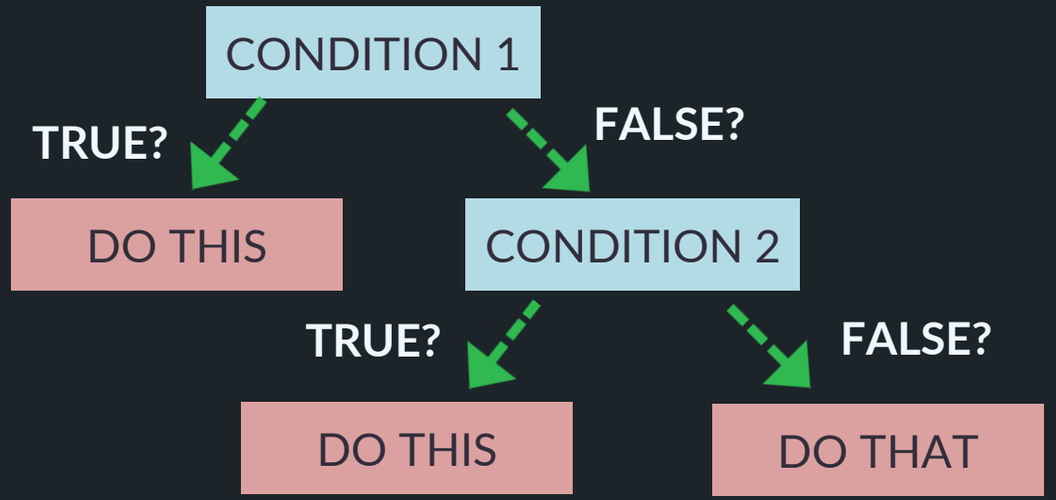
Examples include:

- Waiting for user input
- Detecting a certain type of input
- Knowing when to change theory
- Stopping a process in risk mitigation

Can operate on numeric and non-numeric examples

- Wait for maximum/minimum values
- Select specific entry in database
- Search for existence of entry





CONDITIONAL STATEMENTS: THE IF, ELIF AND ELSE STATEMENTS

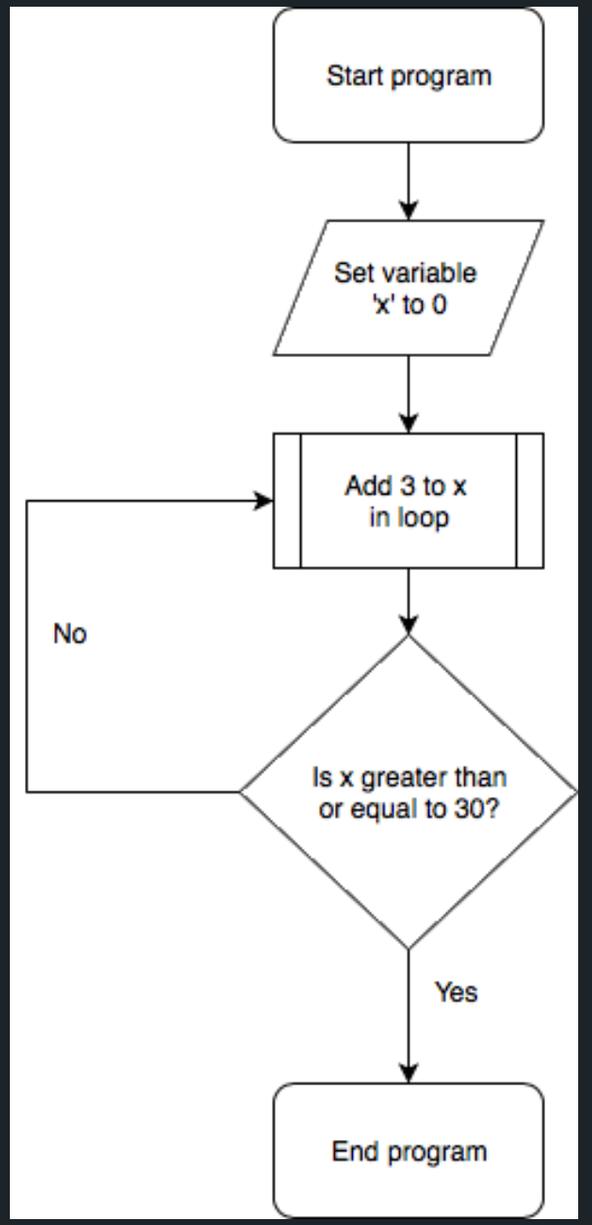
Changing the flow of a program using basic logic very important

Examples include:

- Waiting for user input
- Detecting a certain type of input
- Knowing when to change theory

Can operate on numeric and non-numeric examples

- Wait for maximum/minimum values
- Select specific entry in database
- Search for existence of entry



e.g. two ways to perform the same operation using just IF or combination of IF and ELSE

```
# Initialise two variables with integer values
x=3
y=5
# Now use an IF statement to check the relative values of x and y, then act accordingly
if x > y:
    print("X is greater than Y")
...
if x < y:
    print("X is less than Y")
..."X is less than Y"
```

```
# Initialise two variables with integer values
x=3
y=5
# Now use an IF statement to check the relative values of x and y, th
if x > y:
    print("X is greater than Y")
else x < y:
    print("X is less than Y")
"X is less than Y"
```

Meaning	Math Symbol	Python Symbols
Less than	<	<
Greater than	>	>
Less than or equal	≤	<=
Greater than or equal	≥	>=
Equals	=	==
Not equal	≠	!=

Notice the use of comments using the # symbol and also the correct use of the : operator

e.g. How might we change equation to calculate variable ‘Y’ according to the value of ‘X’

We have two variables 'x' and 'y'. Each has 50 elements and thus we represent in a list, or array. Lets assume that we want to implement two functions:

- one that is used if our x value is less than or equal to 20
- the other if x is greater than 20. We can use a combination of the IF and ELSE statements.

- If X is *less than or equal* to 20, $Y = \frac{X}{12.5}$
- Otherwise [else], $Y = X^{12.5}$

Lets see this implemented as code below. Read through the syntax and if you do not understand, please ask.

Indexing Once again, notice how we have introduced a statement that ends with a colon : and thus requires the next line to be indented.

In [2]:
```
# Initiliase an empty list for both 'x' and 'y'
x = []
y = []

for step in range(50):

    # Append a value to our x array
    x.append(step+1)

    # Now add a conditional statement to check the value of x
    # Notice our additional indentation
    if x[step] <= 20:
        # Append a value to our y array
        y.append(x[step]/12.5)
    else:
        # Append a value to our y array
        y.append(x[step]**12.5)
```

THE 'AND' STATEMENT

Once we move beyond two mutually exclusive conditions, we can also use the ELIF statements. However we need to be careful that we are assigning correct boundaries on our conditions. For example, what if we wanted to check the following conditions:

If variable x < 20, perform action1
If variable x >= 20 but <= 60, perform action2
If variable x > 60, perform action 3

is the following correct?

```
# Version 1
if x[step] < 20:
    <<action>>
elif x[step] >= 20:
    <<action>>
elif x[step] >= 60
    <<action>>
```

THE 'AND' STATEMENT

If variable x < 20, perform action1
If variable x >= 20 but <= 60, perform action2
If variable x > 60, perform action 3

is the following correct?

```
# Version 1
if x[step] < 20:
    <<action>>
elif x[step] >= 20:
    <<action>>
elif x[step] >= 60
    <<action>>

# Version 2
if x[step] < 20:
    <<action>>
elif x[step] >= 20 and x[step] < 60:
    <<action>>
elif x[step] >= 60
    <<action>>
```



PART 2 - LOOPS AND CONDITIONALS



- RECAP AND 'FOR' LOOPS
- CONDITIONAL STATEMENTS
- **NESTED LOOPS [LOOPS IN MORE THAN ONE DIMENSION]**
- PRACTICAL 2



can you help Bob?

David Topping: david.topping@manchester.ac.uk

NESTED LOOPS: WORKING WITH MORE THAN 1 DIMENSION

In many cases we want to work with more than one variable at a time, often in a two [or more] dimensional setting. We can combine 'FOR' loops on any number of levels.

```
for [first iterating variable] in [outer loop]: # Outer loop
    [do something]  # Optional
    for [second iterating variable] in [nested loop]:   # Nested loop
        [do something]
```

We can use conditional statements to refine the process

```
In [10]: # Create two lists of words
         list1 = ['Hello','Goodbye']
         list2 = ['George','Frank','Susan','Sarah']

         for word1 in list1:
             for word2 in list2:
                 if word2 == "Susan":
                     print(word1+' '+word2)

         Hello Susan
         Goodbye Susan
```

NESTED LOOPS: WORKING WITH MORE THAN 1 DIMENSION

In many cases we want to work with more than one variable at a time, often in a two [or more] dimensional setting. We can combine 'FOR' loops on any number of levels.

```
# First loop
for step1 in range(10):
    # Second loop
    for step2 in range(10):
        # Third loop
        for step3 in range(10):
            # Fourth loop
            for step4 in range(10):
                << do domething >>
```

step1



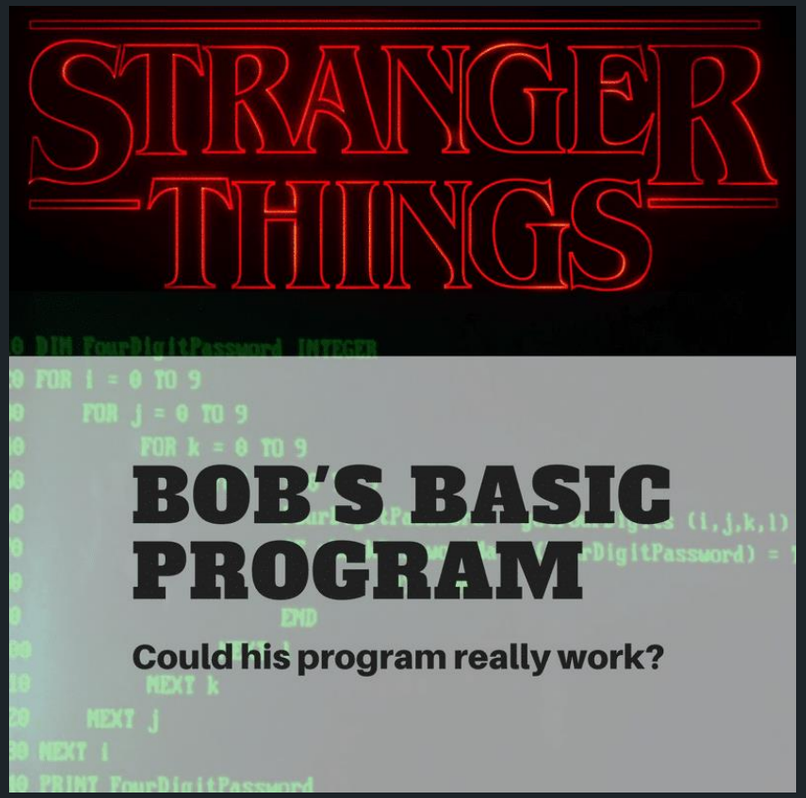
step2



NESTED LOOPS: WORKING WITH MORE THAN 1 DIMENSION

In many cases we want to work with more than one variable at a time, often in a two [or more] dimensional setting. We can combine 'FOR' loops on any number of levels.

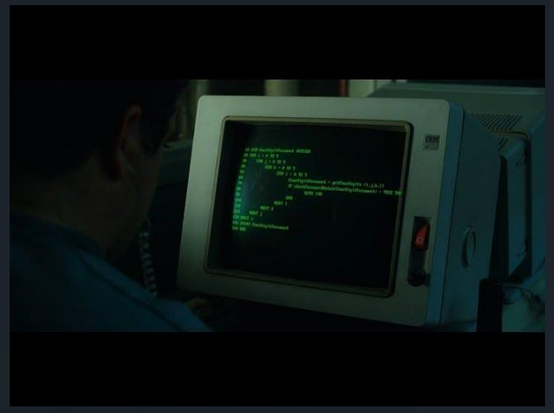
```
10 DIM FourDigitPassword INTEGER
20 FOR i = 0 TO 9
30     FOR j = 0 TO 9
40         FOR k = 0 TO 9
50             FOR l = 0 TO 9
60                 FourDigitPassword = getFourDigits (i,j,k,l)
70                 IF checkPasswordMatch(FourDigitPassword) = TRUE THE
80                     GOTO 140
90                 END
100             NEXT l
110         NEXT k
120     NEXT j
130 NEXT i
140 PRINT FourDigitPassword
150 END
```



PART 2 - LOOPS AND CONDITIONALS



- 'FOR' LOOPS
- CONDITIONAL STATEMENTS
- NESTED LOOPS [LOOPS IN MORE THAN ONE DIMENSION]
- **PRACTICAL 2**



can you help Bob?

David Topping: david.topping@manchester.ac.uk

LECTURE 2

```
# First loop
for step1 in range(10):
    # Second loop
    for step2 in range(10):
        # Third loop
        for step3 in range(10):
            # Fourth loop
            for step4 in range(10):
                << do domething >>
```

```
# Version 1
if x[step] < 20:
    <<action>>
elif x[step] >= 20:
    <<action>>
elif x[step] >= 60
    <<action>>

# Version 2
if x[step] < 20:
    <<action>>
elif x[step] >= 20 and x[step] < 60:
    <<action>>
elif x[step] >= 60
    <<action>>
```

SUMMARY

'FOR' LOOPS

- Allow us to move through lists and arrays sequentially
- Can work with appropriate indices [e.g. x[1]] to add or modify elements
- Can have different 'nested' levels

CONDITIONAL STATEMENTS

- Allow us to check if a condition is true
- Allow us to change the flow of our code
- Work by using a range of operators [e.g. if x == y]
- Working code does not mean it is correct

PRACTICAL 2

```
# First loop
for step1 in range(10):
    # Second loop
    for step2 in range(10):
        # Third loop
        for step3 in range(10):
            # Fourth loop
            for step4 in range(10):
                << do domething >>
```

```
# Version 1
if x[step] < 20:
    <<action>>
elif x[step] >= 20:
    <<action>>
elif x[step] >= 60
    <<action>>

# Version 2
if x[step] < 20:
    <<action>>
elif x[step] >= 20 and x[step] < 60:
    <<action>>
elif x[step] >= 60
    <<action>>
```

GOALS

CODE

- Start to work with FOR loops
- Use conditional statements: IF, ELSE and ELIF
- Nested loops: Start working with more than 1 dimension

GENERAL EXPERIENCE

- Build more complex workflows into your code
- Make code responsive to change of status/input